

Работа с базой данных на основе датасета

**“How Exactly does Literary Content Depend on Genre? A
Case Study of Animals in Children’s Literature”**

Выполнила: Ипатова Оля

Выбор датасета

How Exactly does Literary Content Depend on Genre? A Case Study of Animals in Children's Literature -

<https://dataverse.pushdom.ru/dataset.xhtml?persistentId=doi:10.31860/openlit-2023.10-R005>

О датасете:

- Данные содержат как минимум две сущности для JOIN (авторы и произведения);
- содержат информацию об авторе, произведении;
- информация о животных: лемма, часть речи (словосочетание), контекст употребления

Датасет позволяет исследовать образы животных как культурный и литературный феномен в диахронии.

“Содержание литературного произведения, по крайней мере частично, зависит от литературной традиции. Эта зависимость количественно подтверждается связью жанра с лексическими статистическими закономерностями. Настоящая краткая статья представляет собой шаг на пути к формальному моделированию процессов, определяющих содержание и связанных с литературными жанрами. Идея заключается в том, чтобы объяснить преобладание определённых лемм в литературном тексте с помощью жанрозависимой доступности семантической категории в ходе творческого процесса”

Среда выполнения и подготовка данных

Работу проводила в Google Colab;

Подключение к сети -
`mysql.connector.connect;`

1. Проверила пропуски по столбцам (были только в писателях);
2. удалила строки с пустыми значениями;
3. подвела даты к одному виду;
4. переиндексировала, чтобы не перепутались номера после очистки;

Создание и наполнение таблиц

Таблица авторов:

- `author_id INT AUTO_INCREMENT PRIMARY KEY`
- `author_name VARCHAR(500) NOT NULL UNIQUE`

Наполнение:

- вставляю строки в таблицу, игнорируя дубликаты;
- создаю список кортежей, где каждый кортеж содержит одно значение - имя автора;

Авторов: 942

Таблица произведений:

- `doc_id VARCHAR(500) PRIMARY KEY,`
- `title VARCHAR(1000) NOT NULL,`
- `year INT NOT NULL,`
- `author_id INT NOT NULL,`
- `FOREIGN KEY (author_id) REFERENCES authors(author_id)` - внешний ключ, который ссылается на `author_id` из таблицы `authors`. Это связывает каждое произведение с его автором
- `INDEX idx_year (year)` - создает B-Tree индекс для быстрой фильтрации по году
- `INDEX idx_year_author (year, author_id)` - создает составной индекс для ускорения запросов, где фильтрация идет одновременно по году и автору

Авторы не найдены в словаре: ['Телешов, Николай Дмитриевич']. Произведений: 3015

Создание и наполнение таблиц

Таблица (словарь) с животными:

- animal_id INT AUTO_INCREMENT PRIMARY KEY,
- lemma VARCHAR(500) NOT NULL UNIQUE,
- annotations JSON,
- animal_category VARCHAR(100) -генерируемый столбец из JSON. Он не хранится отдельно, а извлекается из JSON-поля каждый раз при чтении
- INDEX idx_category (animal_category) - индекс по генерируемому столбцу
- FULLTEXT INDEX ft_lemma (lemma) - полнотекстовый поиск

Животных: 1939

Таблица контекстов:

- context_id INT AUTO_INCREMENT PRIMARY KEY,
- doc_id VARCHAR(500) NOT NULL - внешний ключ на таблицу documents
- animal_id INT NOT NULL - внешний ключ на таблицу animals_dictionary
- FOREIGN KEY (doc_id) REFERENCES documents(doc_id)
- FOREIGN KEY (animal_id) REFERENCES animals_dictionary(animal_id)
- INDEX idx_ctx_doc (doc_id) - idx_ctx_doc и idx_ctx_animal создаются для ускорения соединений (JOIN)
- INDEX idx_ctx_animal (animal_id)

Контекстов для вставки: 402267

Готово: 402267 контекстов загружено

Создание словаря животных

```
cursor.execute("""
SELECT lemma, annotations->>'$.category' AS category
FROM animals_dictionary
LIMIT 20
""")
print()
for row in cursor.fetchall():
    print(row)
```

```
cursor.execute("SELECT lemma FROM animals_dictionary ORDER BY lemma")
all_lemmas = [row[0] for row in cursor.fetchall()]
print(all_lemmas)
```

Затем полученные данные и исходную таблицу отправила в ИИ, чтобы быстрее получить словарь по категориям

Работа с индексами и explain

Из таблицы documents временно удаляю индекс, чтобы посмотреть на разницу выполнения запроса.

Формирую запрос через explain:

```
SELECT d.title, d.year, au.author_name  
FROM documents d  
JOIN authors au ON d.author_id = au.author_id  
WHERE d.year BETWEEN 1850 AND 1900  
AND d.author_id = 5
```

```
['id', 'select_type', 'table', 'partitions', 'type',  
'possible_keys', 'key', 'key_len', 'ref', 'rows', 'filtered',  
'Extra']
```

```
(1, 'SIMPLE', 'au', None, 'const', 'PRIMARY', 'PRIMARY',  
'4', 'const', 1, 100.0, None)
```

```
(1, 'SIMPLE', 'd', None, 'ref', 'author_id,idx_year',  
'author_id', '4', 'const', 1, 5.0, 'Using where')
```

Работа с индексами и explain

С индексами

```
['id', 'select_type', 'table', 'partitions', 'type',  
'possible_keys', 'key', 'key_len', 'ref', 'rows', 'filtered',  
'Extra']
```

```
(1, 'SIMPLE', 'au', None, 'const', 'PRIMARY', 'PRIMARY',  
'4', 'const', 1, 100.0, None)
```

```
(1, 'SIMPLE', 'd', None, 'ref',  
'author_id,idx_year,idx_year_author', 'author_id', '4',  
'const', 1, 5.0, 'Using where')
```

Без индексов

```
['id', 'select_type', 'table', 'partitions', 'type',  
'possible_keys', 'key', 'key_len', 'ref', 'rows', 'filtered',  
'Extra']
```

```
(1, 'SIMPLE', 'au', None, 'const', 'PRIMARY', 'PRIMARY',  
'4', 'const', 1, 100.0, None)
```

```
(1, 'SIMPLE', 'd', None, 'ref', 'author_id,idx_year',  
'author_id', '4', 'const', 1, 5.0, 'Using where')
```

*если бы у автора было много книг, можно было бы выбрать
idx_year_author, потому что он позволяет сразу отфильтровать и по году,
и по автору, не делая дополнительной фильтрации

Запросы

SELECT с фильтрацией и сортировкой. Произведения до 1917 года. Использует B-tree индекс `idx_year`.

JOIN топ-10 животных по числу упоминаний;

Подзапрос: произведения с числом упоминаний выше среднего по корпусу

GROUP BY + HAVING + RANK: Авторы по разнообразию животных с рангом внутри исторической эпохи.

ROW_NUMBER: хронологическая нумерация произведений каждого автора

JSON_TABLE: разворачиваем JSON-поле `meta` в строки. Использую JSON_TABLE для того, чтобы развернуть JSON-данные из столбца `meta` в реляционные строки, как если бы они были в отдельных столбцах

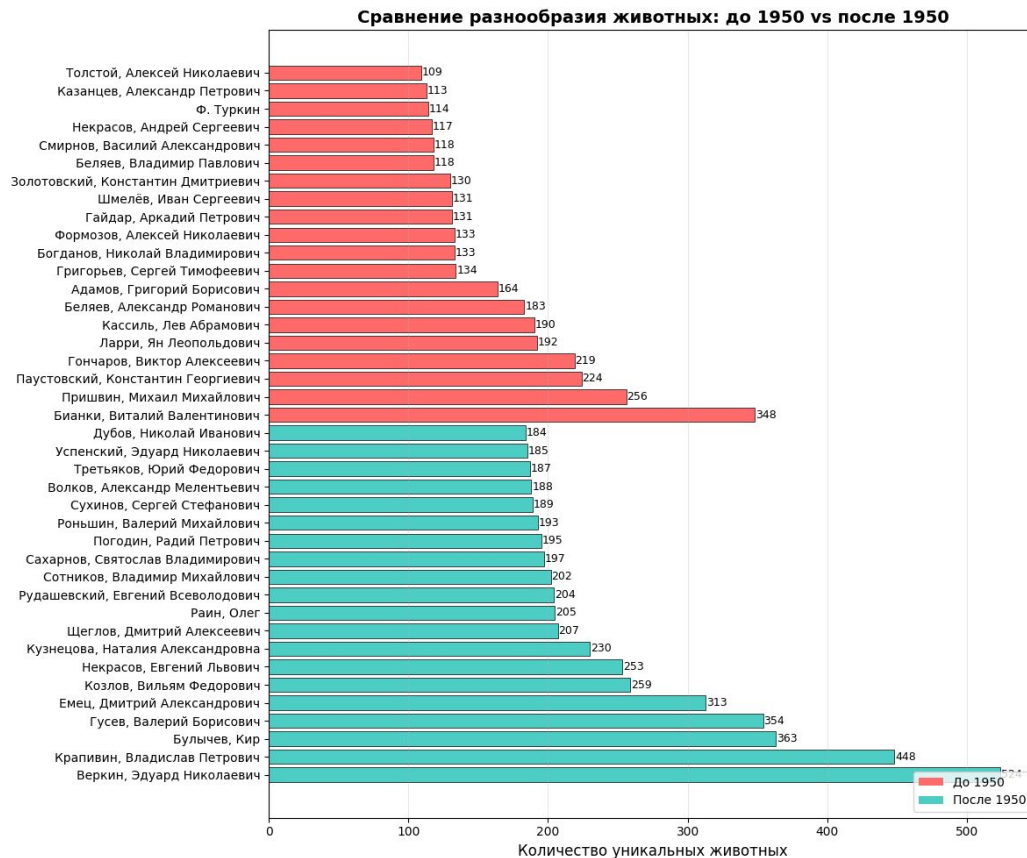
JSON_SET: обновление JSON-документа внутри запроса

Поиск MATCH AGAINST: запрос показывает леммы, релевантные запросу "птица медведь кот", и выводит релевантность

JSON_TABLE: разворачиваем JSON-поле `meta` в строки.

Использую JSON_TABLE для того, чтобы развернуть JSON-данные из столбца `meta` в реляционные строки, как если бы они были в отдельных столбцах

Сравнение разнообразия животных у авторов по эпохам



Динамика упоминаний по годам



Динамика упоминаний “коровы” по годам



Динамика упоминаний “собаки” по годам



Выводы

Благодаря подробной работе с базами данных, удалось подробно изучить датасет и внести новые (сгенерированный столбик с категориями животных);

Удалось успешно поработать с запросами и освоить новые (match against rank);

Свой анализ провела в основном благодаря анализу частотности лемм и категорий.

Трудности:

были проблемы с очисткой данных на разных этапах, приходилось несколько раз перезапускать и удалять созданные таблицы, поэтому приняла решение очистить все столбцы. С одним автором также были проблемы в сыром датасете, пришлось удалить некоторые данные по автору в целом.